

Problem 1: Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;
    int s =1;
    while(s <= n)
    {
        i++;
        s += i;
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

ANSWER

```
#include <stdio.h>
int main()
{
    int n;
    int i = 1;
    int s = 1;
    int count = 0;
    scanf("%d", &n);
    count+=2;
    while (s <= n)
    {
        count++;
        i++;
        count++;
        s += i;
        count++;
    }
    count++;
    printf("%d", count);
    return 0;
}
```

	Input	Expected	Got	
	9	12	12	
	4	9	9	

Passed all tests!

Correct

Problem 2: Finding Complexity using Counter method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
        printf("*");
    }
    else
    {
        for(int i=1; i<=n; i++)
        {
            for(int j=1; j<=n; j++)
            {
                printf("*");
                printf("*");
                break;
            }
        }
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```
#include <stdio.h>
int main()
{
    int n, count = 1;
    scanf("%d", &n);
    if (n == 1)
    {
        count++;
    }
    else
    {
        int i, j;
        for (i = 1; i <= n; i++)
        {
            count++; // i <= n
            for (j = 1; j <= n; j++)
```

```

    {
        count++;
        count++;
        count++;
        break;
    }
    count++;
}
count++;
}
printf("%d", count);
return 0;
}

```

	Input	Expected	Got	
	2	12	12	
	1000	5002	5002	
	143	717	717	

Passed all tests!

Correct

Problem 3: Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time complexity using counter method.

```

Factor(num) {
{
    for (i = 1; i <= num; ++i)
    {
        if (num % i == 0)
        {
            printf("%d ", i);
        }
    }
}
}

```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

ANSWER:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, count = 0;
```

```
    scanf("%d", &n);
```

```
    int i;
```

```

for (i = 1; i <= n; ++i)
{
    count++;
    count++;
    if (n % i == 0)
    {
        count++;
    }
}
count++;
printf("%d", count);
return 0;
}

```

	Input	Expected	Got	
	12	31	31	
	25	54	54	
	4	12	12	

Passed all tests!

Correct

Problem 4: Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time complexity using counter method.

```

void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}

```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,j,k;
```

```

int c=2;
scanf("%d",&n);
for(i=n/2;i<n;i++){
    c+=2;
    for(j=1;j<n;j=2*j){
        c+=2;
        for(k=1;k<n;k=k*2)
        {
            c+=2;
        }
    }
}
printf("%d",c);
return 0;
}

```

Feedback

Input	Expected	Got	
4	30	30	
10	212	212	

Passed all tests!

Correct

Problem 5: Finding Complexity using counter method

Convert the following algorithm into a program and find its time complexity using counter method.

```

void reverse(int n)
{
    int rev = 0, remainder;
    while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;
    }
    print(rev);
}

```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

ANSWER

```
#include<stdio.h>
```

```

int main()
{
    int n, rev = 0, remainder;
    int count = 2;
    scanf("%d", &n);
    while (n != 0)
    {
        count++;
        remainder = n % 10;
        count++;
        rev = rev * 10 + remainder;
        count++;
        n /= 10;
        count++;
    }
    count++;
    printf("%d", count);
    return 0;
}

```

	Input	Expected	Got	
	12	11	11	
	1234	19	19	

Passed all tests!

Correct